Designing Microservices: Responsibilities, APIs and Collaborations - workshop overview

Chris Richardson

Founder of Eventuate.io

Founder of the original CloudFoundry.com

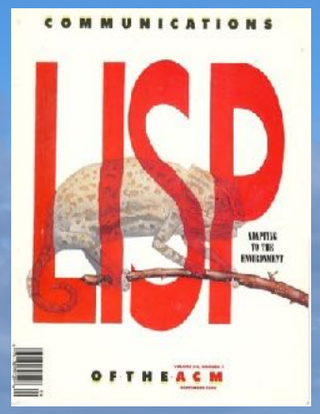
Author of POJOs in Action and Microservices Patterns

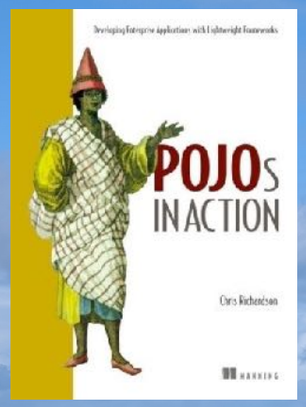
@crichardson

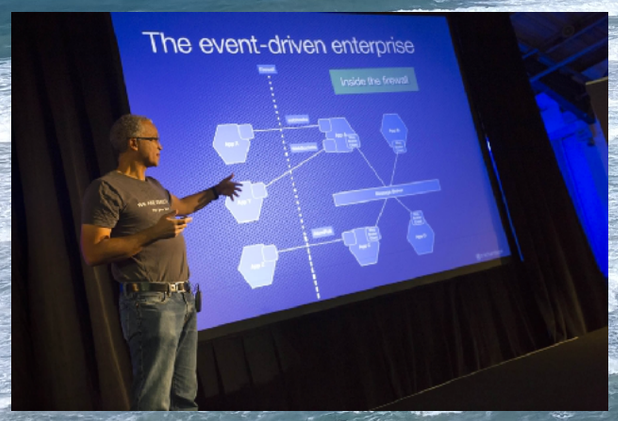
chris@chrisrichardson.net

adopt.microservices.io

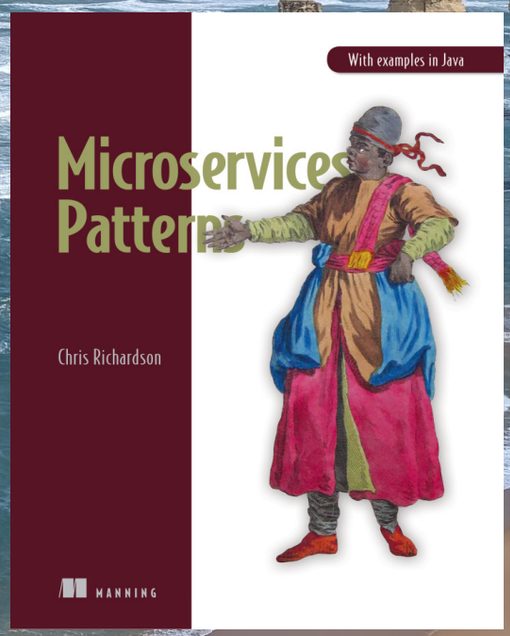
About Chris







eventuate



http://adopt.microservices.io

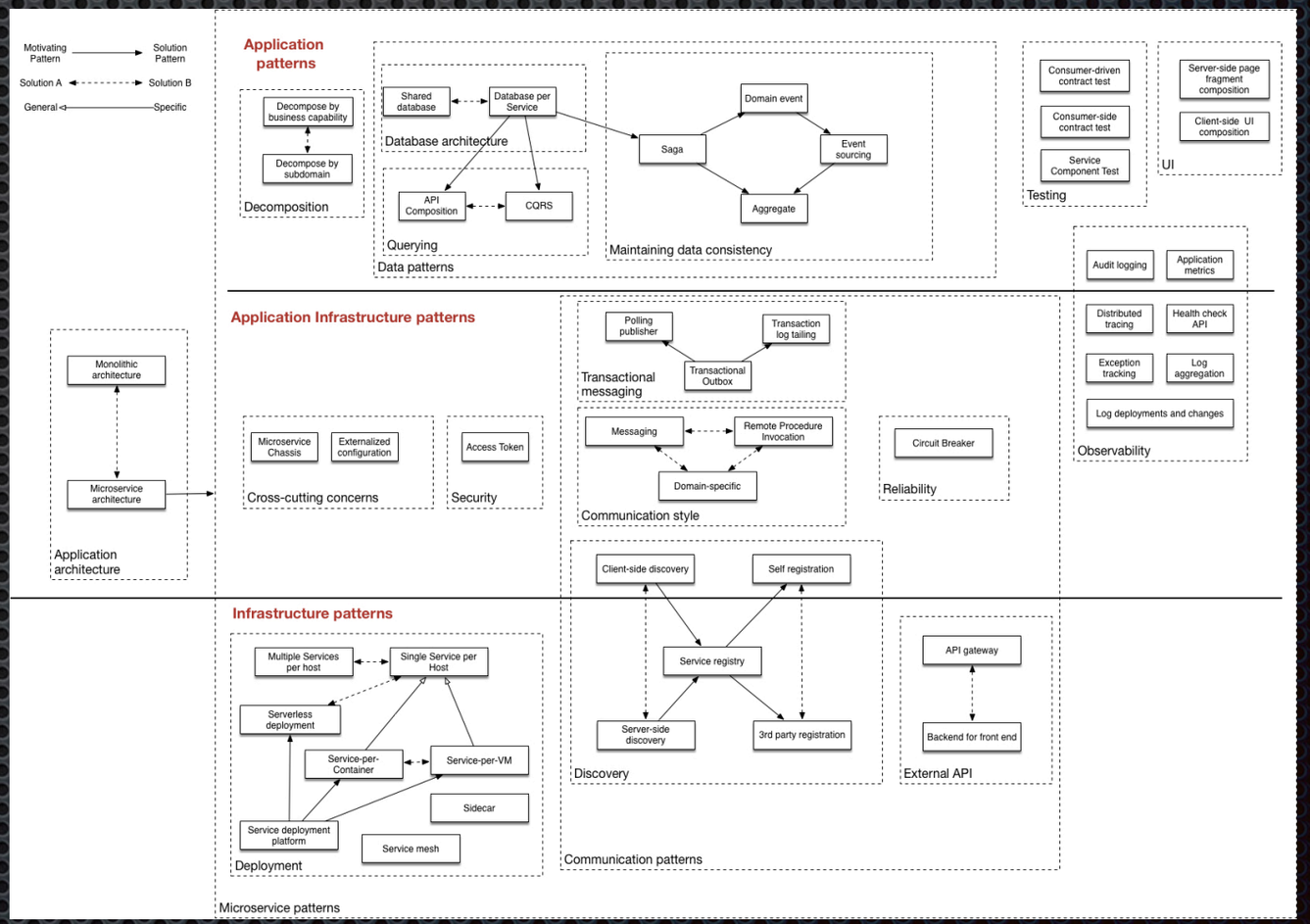
About Chris

https://chrisrichardson.net/

@crichardson

About Chris: microservices.io

- Microservices pattern language
- Articles
- Code examples



adopt.microservices.io

Who are you?

Workshop overview: The architecture definition process

The starting point

0

Functional requirements

As a consumer
I want to place an Order
So that I can ….

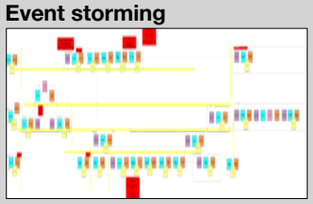
As a Restaurant
I want to accept an Order
So that I can ….

System quality attributes

- SLA: Reliability/Latency
- Scalability
- …

Wireframe/UI mockups

Event storming



The process

4 **Evaluate** 5 **Refactor**

1 **Define system operations**

2 **Define subdomains**

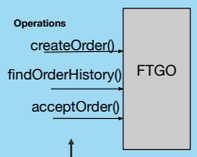
3 **For each operation:**

Design operation and services

Group subdomains

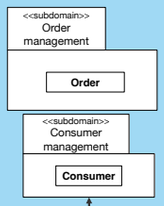
Design collaborations

System operations



Represents an external request

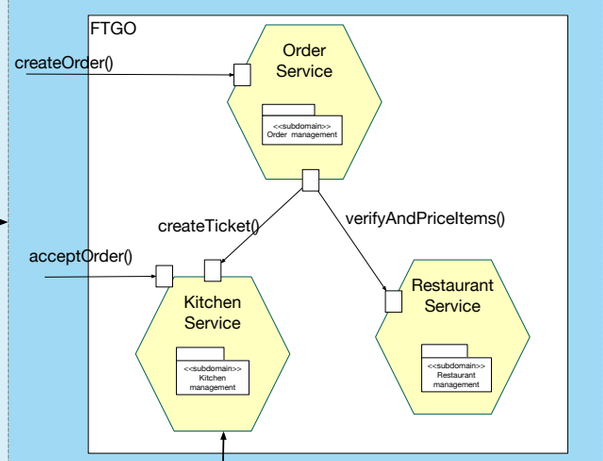
Subdomains



Models a business capability

The result

Service architecture



Service = group of subdomains

Workshop overview

- Architecting for modern software delivery
- Overview of Assemblage - the microservice architecture definition process
- Discovering system operations
- Defining subdomains
- Designing system operations: commands and queries
- Evaluating and refactoring a microservice architecture

1. Lectures
2. Katas
3. Kata reviews

About the katas/design exercises

- Define a microservice architecture for the Marigold Hotel application step-by-step
- Work together in small groups
- Present and discuss results in a review session

Documentation templates

https://bit.ly/dddeu-2025-msa-workshop

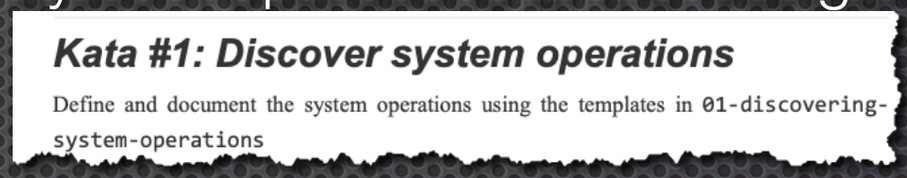
Agenda: Day 1

Workshop overview

Discussion — Q&A about homework videos

Lectures
- Architecting for modern software delivery
- Overview of Assemblage
- Discovering system operations

Kata — Discover system operations of the Marigold Hotel application



Kata review — System operations of the Marigold Hotel application

Lecture — Designing subdomains

Kata — Discover subdomains of the Marigold Hotel application

@crichardson chris@chrisrichardson.net

Questions?

Let's make this interactive!

adopt.microservices.io

Agenda: Day 2

Kata review — Subdomains of the Marigold Hotel application

Lecture — Designing system operations: commands and queries

Kata — Designing system operations: commands and queries

Kata #3: Rank system operations

Rank the system operations using the template in 03-01-ranking-system-operations

Kata #4: Design system queries

Kata #5: Design system commands

Kata review — Designing system operations: commands and queries

Lecture — Evaluating and refactoring a microservice architecture

About:

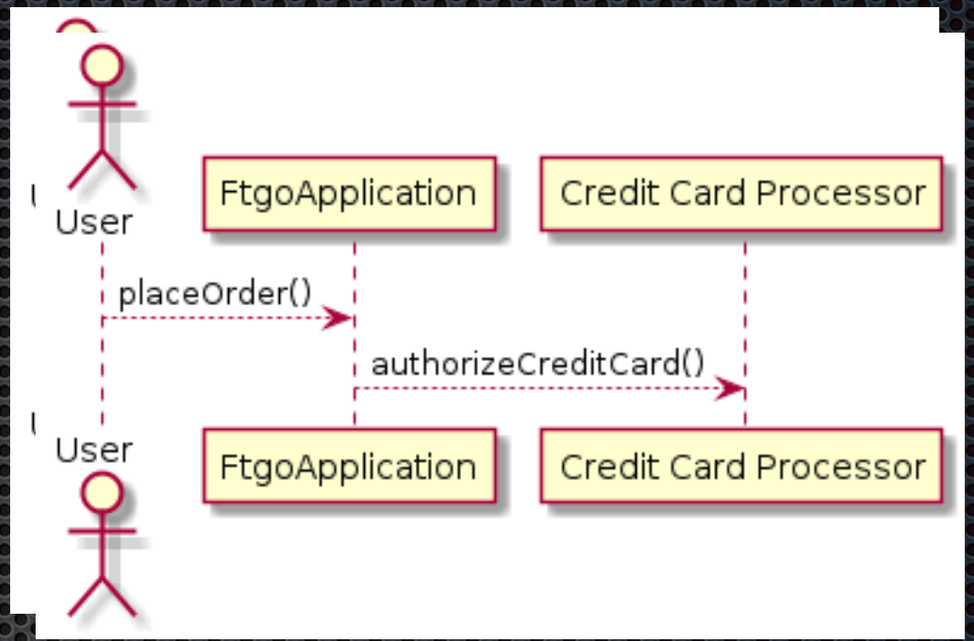
Kata #1: Discover system operations

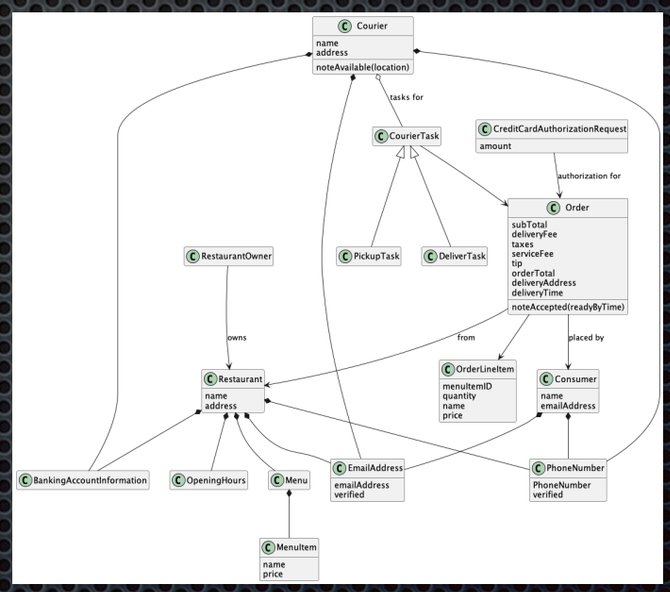
Define and document the system operations using the templates in `01-discovering-system-operations`

- As a group:
  - Identify *first* system **command**
  - Create specification
- As a group:
  - Brainstorm system operations
  - Divide amongst pairs (or maintain a task queue)
- In pairs: document *each* assigned system operation
- As a group: Review and *harmonize* system operation definitions

Summary: deliverables

System command canvas	
Name	`placeOrder(consumerID, deliveryTime, deliveryAddress, restaurantID, lineItems)`
Parameters	• `consumerID` - the ID of the consumer placing the order • `deliveryTime` - the desired delivery time • `deliveryAddress` - the delivery address • `restaurantID` - the restaurant • `lineItems` - collection of `OrderLineItem(menuItemIDs, quantity)`
Return value	• `orderID`
Trigger	User command
Service Level Objectives	• Business tier: Critical, Core, User, Internal • Throughput, e.g. requests/second • Availability - ... • Latency - ... • ...
Preconditions	• a Consumer with ID `consumerID` exists • the Consumer has a valid payment method • a Restaurant with ID `restaurantID` exists • the `deliveryAddress` and `deliveryTime` are served by the restaurant • `MenuItem`'s identified by `menuItemID` exists
Postconditions	• an `Order order` was created in the `APPROVED` state • a `CreditCardAuthorization` for the `orderTotal` for the Consumer's credit card was created • `returnValue` became `order.ID`






```
Chriss-iMac-Pro:templates cer$ unzip -l microservice-architecture-design-templates-082022.zip '*01-discovering-system-operations*'
Archive:  microservice-architecture-design-templates-082022.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
   166460  08-19-2022 21:13   pdfs/01-discovering-system-operations.pdf
   107571  08-19-2022 21:45   docx/01-discovering-system-operations.docx
     3362  08-19-2022 21:02   01-discovering-system-operations.adoc
---------                     -------
   277393                     3 files
```

Useful templates

About:

Kata #2: Define subdomains

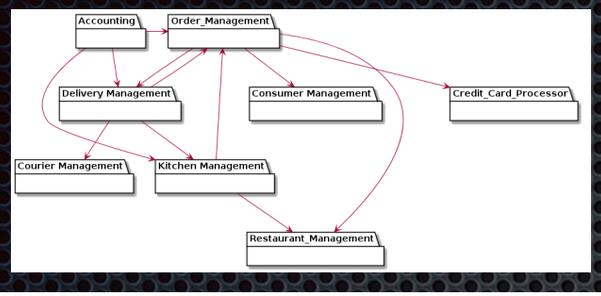
Define and document the subdomains using the templates in `02-defining-subdomains`

- As a group:
  - Discover and define subdomains
  - Assign system operations to pairs (or maintain a task queue)
- In pairs:
  - Design each system operation collaborations in terms of entities and subdomains
- As a group:
  - Review and *harmonize* system operation collaborations
  - Refine subdomains

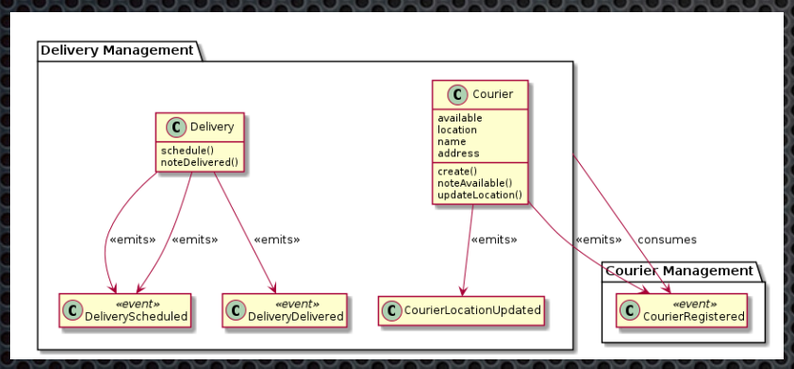
Key model behavior

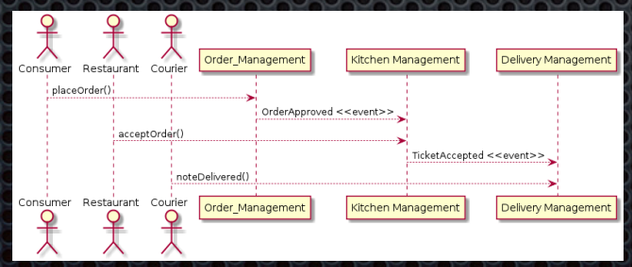
- Keep track of the number of rooms of each room type that are available for sale on a given date
- Handle the following *make reservation* scenario:
  - Only one room of a room type available at a given price
  - Two simultaneous requests for the same room type
  - One should succeed, one should fail (get higher price)

Summary: deliverables



Subdomain canvas	
Name	Restaurant Management
Responsibilities	• Manages information about restaurants • Knows restaurant name, location, description, opening hours, and menu
Collaborations	• External address validation and geocoding service
Type	Supporting
Complexity	Simple
Key aggregates	• Restaurant





Revised

System command canvas	
Name	`placeOrder(consumerID, deliveryTime, deliveryAddress, restaurantID, lineItems)`
Parameters	• `consumerID` - the ID of the consumer placing the order • `deliveryTime` - the desired delivery time • `deliveryAddress` - the delivery address • `restaurantID` - the restaurant • `lineItems` - collection of `OrderLineItem(menuItemIDs, quantity)`
Return value	• `orderID`
Trigger	User command
Service Level Objectives	• Business tier: Critical, Core, User, Internal • Throughput, e.g. requests/second • Availability - ... • Latency - ... • ...
Preconditions	• a Consumer with ID `consumerID` exists • the Consumer has a valid payment method • a Restaurant with ID `restaurantID` exists • the `deliveryAddress` and `deliveryTime` are served by the restaurant • `MenuItem`'s identified by `menuItemID` exists
Postconditions	• an `Order order` was created in the `APPROVED` state • a `CreditCardAuthorization` for the `orderTotal` for the Consumer's credit card was created • `returnValue` became `order.ID`

Template: 02-defining-subdomains.json

About

Kata #3: Rank system operations

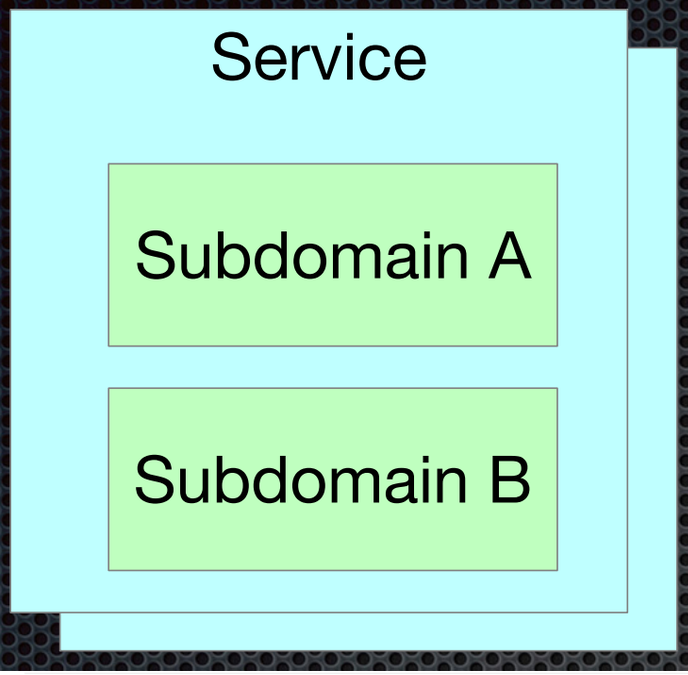
Rank the system operations using the template in 03-01-ranking-system-operations

Kata #4: Design system queries

Kata #5: Design system commands

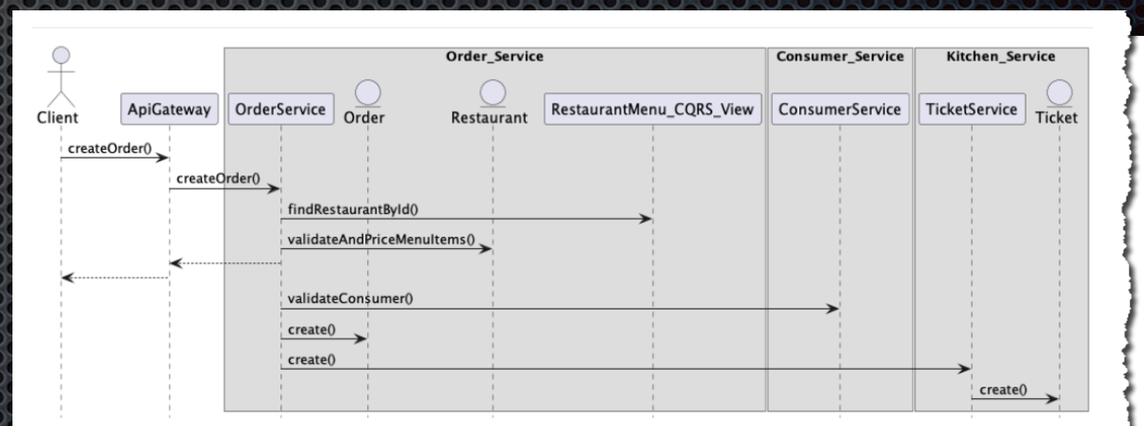
- As a group:
  - Rank system operations spanning multiple subdomains by importance
    - For each operation
      - If it spans multiple services
        - Design collaboration and assign subdomains to services
- In pairs: create sequence diagram for assigned operations

Summary: Deliverables…



System command design canvas	
Name:	createOrder()
Entry point:	
Service:	Order Service
Operation:	createOrder()
Collaboration patterns:	
Sagas	• `Create Order`
Command-side Replicas	• `Restaurant Menu View`

Service canvas		
Name:	Order Service	
Description:		
Responsibilities	• Manages orders	
Subdomains	• Order management	
Service API		
Commands	Queries	Events Published
• createOrder()	• findOrder()	• OrderCreated
Non-functional requirements		
Dependencies		
Invokes	Subscribes to	
	Restaurant Service: • Restaurant Created • Restaurant Updated • Restaurant …	



Templates:

- 03-01-ranking-system-operations
- 03-02-designing-operations-and-services

...Summary: deliverables

System query design canvas	
Name:	findOrder()
Entry point:	
Service:	API Gateway
Operation:	
Collaboration	
API Composit	
CQRS	

API composition canvas	
Query:	findOrder(param1, param2,)
Parameters:	TBD
Returns:	TBD
Strategy:	API Composition
Composer:	API Gateway
Providers	
Order Service	
Delivery Service	
Kitchen Service	
Delivery Service	
...	

Query:	findOrder()
Parameters:	TBD
Returns:	TBD
Strategy:	CQRS
View Service:	Order History Service
View design	
Database type:	TBD
Schema design:	TBD
Command-side services	
Service name	Events
Order Service	• Order Created • Order Canceled
Delivery Service	• Delivery scheduled • Picked up